# Bézier Curve: Generic + Symbolic

GLMakie + Symbolics.jl

Code and Plot Explanation for Julia Beginners

`32-bezier-symbolic.jl`

E.R. Nurwijayadi

2026-03-15

*Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.*

This document explains `32-bezier-symbolic.jl`. It is the symbolic counterpart to `31-bezier-generic.jl`: the same generic curve and drawing code, with one difference — curve points are computed via **Symbolics.jl** instead of direct Bernstein evaluation. The polynomial is built once as a symbolic expression, then evaluated by substitution. This document focuses entirely on what changed. Everything else is documented in `31-bezier-generic-explained`.

---

## Contents

**1 The Two Plots** **2**
1.1 Case 1 — Quadratic 2-D . . . . . 2
1.2 Case 2 — Quartic 3-D . . . . . 3

**2 What Is Identical to Script 31** **3**

**3 What Changed: The Symbolic Engine** **4**

**4 The Symbolic Variable** **4**

**5 `build_symbolic_poly` — One Polynomial per Dimension** **4**

**6 `bz_point` — Evaluation by Substitution** **5**

**7 `main()` — Printing the Polynomials** **6**

**8 The Equation Label** **7**

**9 Summary: Script 31 vs Script 32** **7**

# 1. The Two Plots

## 1.1. Case 1 — Quadratic 2-D

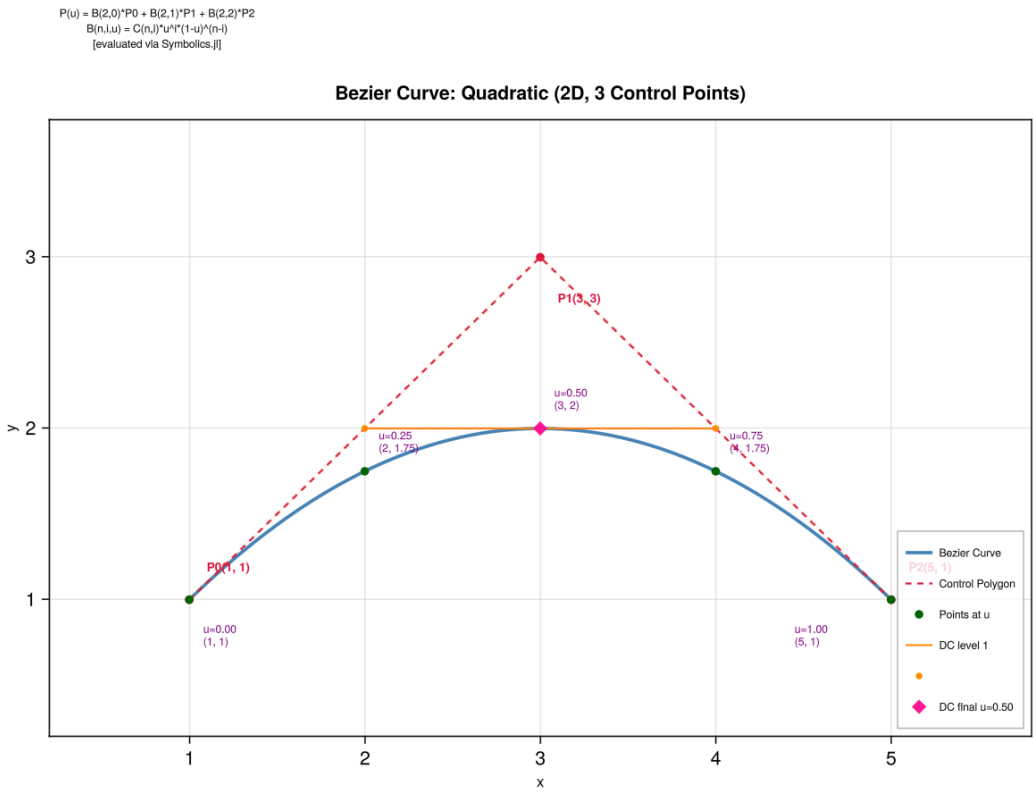


Figure 1: Case 1: quadratic Bézier in 2-D. Note the equation label now includes `[evaluated via Symbolics.jl]`.

### 1.2. Case 2 — Quartic 3-D

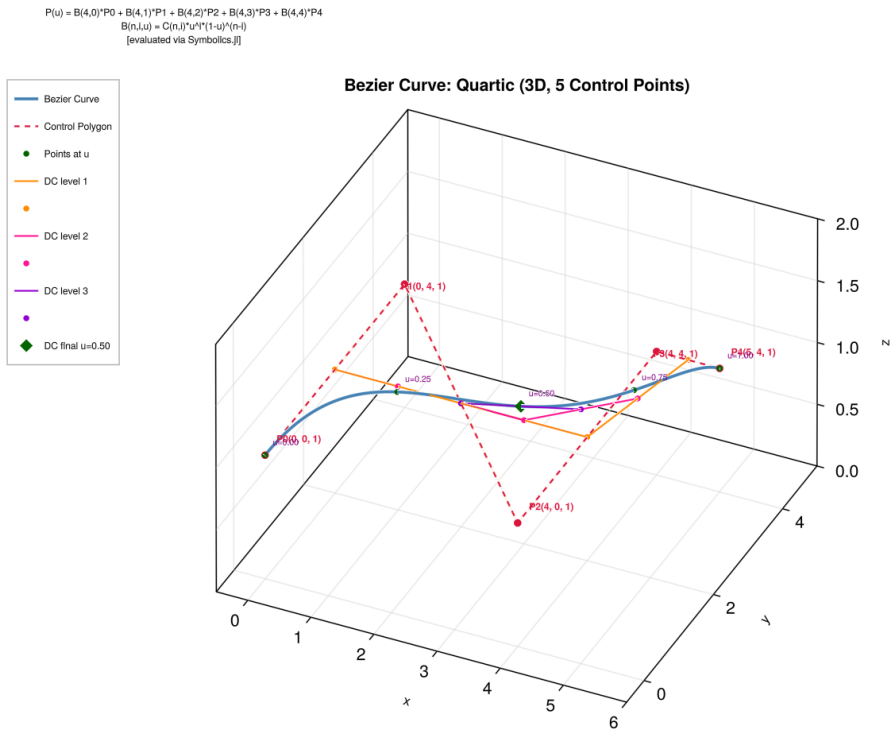


Figure 2: Case 2: quartic Bézier in 3-D. Visually identical to script 31.

The plots are visually identical to script 31. The only visible difference is the third line in the equation label: `[evaluated via Symbolics.jl]`.

## 2. What Is Identical to Script 31

| Component | Note |
|---|---|
| `CASE_DATA` | Identical — same points, $u$ values, filenames. |
| `BezierCurve` struct & constructor | Identical. |
| `degree_name`, `dim_name` | Identical. |
| `de_casteljau` | Identical — generic `while` loop. |
| All colour constants | Identical. |
| `draw_and_save` body | Identical — same `is_3d` branching, same drawing layers, same De Casteljau loop, same auto-limits, same legend. |
| `Label` equation box | Nearly identical: one added line `[evaluated via Symbolics.jl]`. |

## 3. What Changed: The Symbolic Engine

*Script 31 used a simple numeric Bernstein formula. Script 32 replaces that with a two-step approach: build the polynomial symbolically once, then evaluate by substituting u values.*

| Component | Script 31 | Script 32 |
|---|---|---|
| Symbolic variable | none | `@variables u_sym` |
| Polynomial construction | none | `build_symbolic_poly(bz)` |
| `bz_point` signature | `(bz, u)` | `(bz, polys, u)` |
| Evaluation method | Direct Bernstein formula | `substitute(p, Dict(u_sym` |
| `bz_curve` / `bz_highlight` | `(bz, n_pts)` | `(bz, polys, n_pts)` |
| `draw_and_save` signature | `(bz, u_highlight, u_dc, filename)` | `(bz, polys, u_highlight,` |
| `main()` | Direct call | Builds `polys`, prints them, t |

## 4. The Symbolic Variable

```
@variables u_sym
```

Creates a symbolic variable named `u_sym` using Symbolics.jl. It is named `u_sym` rather than `u` to avoid any possible collision with local numeric variable names inside loops. This symbolic variable is declared at module level — outside all functions — so every function in the script can use it.

> **Why `u_sym` and not `u`?** In scripts 12 and 22, `@variables u` declared a symbolic `u`, which caused loop variables in comprehensions to be renamed to `u_val` to avoid shadowing. Using `u_sym` from the start sidesteps this naming conflict cleanly — numeric loop variables can freely use `u` without any risk of confusion.

## 5. `build_symbolic_poly` — One Polynomial per Dimension

```
function build_symbolic_poly(bz::BezierCurve)
    n = bz.degree
    [
        sum(
            binomial(n, i-1) * u_sym^(i-1) * (1 - u_sym)^(n-(i-1))
              * bz.points[i, d]
            for i in 1:bz.n
        )
        for d in 1:bz.dim
    ]
end
```

This function builds the Bézier polynomial as a symbolic expression for each spatial dimension.

**Structure:** a nested list comprehension. The outer loop `for d in 1:bz.dim` iterates over dimensions ($x$, $y$, and optionally $z$). The inner `sum(...for i...)` accumulates the weighted Bernstein terms for that dimension.

**Each term:** `binomial(n, i-1)` is the binomial coefficient — an exact integer from Julia's built-in function. `u_sym^(i-1)` is a symbolic power of `u_sym`. `(1 - u_sym)^(n-(i-1))` is another symbolic power. `bz.points[i, d]` is the scalar coordinate — a `Float64` that gets multiplied into the symbolic expression.

The result is a `Vector` of symbolic expressions, one per dimension. This vector is computed *once* and passed to all evaluation functions.

**For Case 1 (quadratic 2-D), `build_symbolic_poly` returns:**

`polys[1]` (the $x$ polynomial, before expansion):

$\binom{2}{0}(1-u_{\text{sym}})^2 \cdot 1 + \binom{2}{1} u_{\text{sym}}(1-u_{\text{sym}}) \cdot 3 + \binom{2}{2} u_{\text{sym}}^2 \cdot 5$

After simplification by Symbolics.jl: $x(u) = 1 + 4u$

`polys[2]` (the $y$ polynomial, before expansion):

$\binom{2}{0}(1-u_{\text{sym}})^2 \cdot 1 + \binom{2}{1} u_{\text{sym}}(1-u_{\text{sym}}) \cdot 3 + \binom{2}{2} u_{\text{sym}}^2 \cdot 1$

After simplification: $y(u) = -4u^2 + 4u + 1$

Both are stored as *unevaluated* Symbolics.jl expressions — no numeric $u$ has been substituted yet.

**Why build the polynomial once and reuse it?**

In script 31, every call to `bz_point(bz, u)` computed $\sum B(n, i, u) \cdot P_i$ from scratch — all the powers, binomials, and multiplications, for every single $u$.

In script 32, `build_symbolic_poly` constructs the polynomial expression once. Evaluating at 300 $u$ values for the curve then requires 300 substitution calls on the pre-built expression, rather than 300 fresh Bernstein computations. The symbolic representation encodes the full polynomial structure, and substitution is a simple variable replacement.

For a quartic 3-D curve with 5 control points and 300 sample points, this means building 3 symbolic expressions once, then doing 900 substitutions, rather than $300 \times 5 \times 3 = 4500$ full Bernstein computations.

## 6. bz_point — Evaluation by Substitution

```
function bz_point(bz::BezierCurve, polys::Vector, u::Float64)
    [Float64(Symbolics.value(substitute(p, Dict(u_sym => u))))
     for p in polys]
end
```

For each symbolic polynomial `p` in `polys`, this substitutes the numeric value `u` for the symbolic variable `u_sym` and extracts a `Float64` result.

**`substitute(p, Dict(u_sym => u))`:** Replaces every occurrence of `u_sym` in the expression `p` with the numeric value `u`. Returns a Symbolics.jl expression that simplifies to a number (but is still wrapped in the Symbolics type).

**`Symbolics.value(...)`**: Unwraps the Symbolics wrapper and returns the underlying numeric value (a `SymbolicUtils` numeric type).

**`Float64(...)`:** Converts the result to a plain Julia `Float64`.

**Evaluating $x(0.5)$ for Case 1:**

`p = polys[1]` represents $1 + 4u_{\text{sym}}$.

`substitute(p, Dict(u_sym => 0.5))` $\rightarrow$ Symbolics expression for $1+4\times0.5 = 3$

`Symbolics.value(...)` $\rightarrow$ `3` (numeric)

`Float64(3)` $\rightarrow$ `3.0`

The full `bz_point` call for $u = 0.5$ returns `[3.0, 2.0]` — same as script 31. ✓

**Compared to script 31's `bz_point`:**

Script 31:

```
function bz_point(bz::BezierCurve, u::Float64)
    sum(bernstein(bz.degree, i-1, u) .* bz.points[i,:]
        for i in 1:bz.n)
end
```

Script 32:

```
function bz_point(bz::BezierCurve, polys::Vector, u::Float64)
    [Float64(Symbolics.value(substitute(p, Dict(u_sym => u))))
     for p in polys]
end
```

Script 31 takes two arguments (`bz` and `u`) and computes the Bernstein formula directly. Script 32 takes three arguments (`bz`, `polys`, and `u`) and evaluates prebuilt symbolic expressions. Both return a length-`dim` vector of coordinates. The `polys` argument propagates through `bz_curve` and `bz_highlight` in the same way.

## 7. `main()` — Printing the Polynomials

```
function main()
    data  = CASE_DATA[CASE]
    bz    = BezierCurve(data.points)
    println("Building symbolic polynomials...")
    polys = build_symbolic_poly(bz)
    println("Polynomials:")
    for (d, axis) in enumerate(bz.dim == 3 ? ["x","y","z"] : ["x","y"])
        println("  $(axis)(u) = $(expand(polys[d]))")
    end
    draw_and_save(bz, polys, data.u_highlight, data.u_dc, data.output)
end
```

Script 32's `main` does something script 31 did not: it prints the symbolic polynomials to the terminal before drawing the plot.

**`expand(polys[d])`:** Calls Symbolics.jl's `expand()` on the symbolic expression to fully distribute and collect terms. This produces the clean monomial form for display — the same polynomials computed algebraically in scripts 01–05.

**The axis name list:** `bz.dim == 3 ? ["x","y","z"] : ["x","y"]` is a ternary expression returning the appropriate axis name list for 2-D or 3-D.

**Terminal output for Case 1 (`CASE=1`):**

```
Building symbolic polynomials...
Polynomials:
  x(u) = 1 + 4u_sym
  y(u) = 1 + 4u_sym - 4(u_sym^2)
```

**Terminal output for Case 2 (`CASE=2`):**

```
Building symbolic polynomials...
Polynomials:
  x(u) = (24//1)*(u_sym^2) - (32//1)*(u_sym^3) + (13//1)*(u_sym^4)
  y(u) = (16//1)*u_sym - (48//1)*(u_sym^2) + ...
  z(u) = 1
```

These are the exact same polynomials from scripts 01–05, now derived symbolically from the control points. The `u_sym` in the output is the name of the symbolic variable as declared.

## 8. The Equation Label

```
eq = "P(u) = $terms\nB(n,i,u) = C(n,i)*u^i*(1-u)^(n-i)\n[evaluated via
↪  Symbolics.jl]"
```

The only visible difference in the plots is this third line in the equation label: `[evaluated via Symbolics.jl]`. It signals to the reader that the curve was not computed by direct Bernstein arithmetic but through symbolic polynomial evaluation. The label is otherwise identical to script 31.

## 9. Summary: Script 31 vs Script 32

| Aspect | Script 31 | Script 32 |
|---|---|---|
| Evaluation engine | Direct Bernstein formula per point | Symbolic polynomial built once, then substituted |
| Symbolic variable | None | `@variables u_sym` |
| Key new function | `bernstein(n,i,u)` | `build_symbolic_poly(bz)` |
| `bz_point` method | `(bz, u)` | `(bz, polys, u)` |
| Terminal output | None beyond progress messages | Prints the expanded polynomial per axis |
| Equation label | Two lines | Three lines: adds `[evaluated via Symbolics.jl]` |
| Plot appearance | Identical to script 32 | Identical to script 31 |

**Which should you use?**

Script 31 is simpler and faster to understand — the evaluation is explicit arithmetic you can trace by hand.

Script 32 is more powerful for exploration — `build_symbolic_poly` gives you the full polynomial expression that you can inspect, differentiate, or manipulate before evaluating. The terminal printout of the polynomials is itself useful: it confirms that the control points produce the expected algebraic form.

*Script 31 computes the curve. Script 32 also* understands *it.*